

OAuth for Youtube

Google Cloud Console Setup

Step 1: Create a Project

1. Go to [Google Cloud Console](#)
2. Click **"Select a Project"** → **"New Project"**
3. Name your project → Click **"Create"**

Step 2: Enable YouTube Data API

1. Navigate to **"APIs & Services"** → **"Library"**
2. Search for **"YouTube Data API v3"**
3. Click **"Enable"**

Step 3: Configure OAuth Consent Screen

1. Go to **"APIs & Services"** → **"OAuth consent screen"**
2. **User Type:** Select **"External"** (for testing)
3. Fill in:
 - App name (anything you want)
 - Support email
 - Developer email
4. Click **"SAVE AND CONTINUE"**

Add Scopes

Data Access → Click **"ADD OR REMOVE SCOPES"** → Search for "YouTube" → Select required scopes:

Scope	Purpose
<code>.../auth/youtube</code>	Full account management
<code>.../auth/youtube.upload</code>	Upload videos only
<code>.../auth/youtube.readonly</code>	View-only access

Scope	Purpose
<code>.../auth/youtube.force-ssl</code>	Full access with SSL

Add Test Users

1. Audience → Scroll to **"Test users"** section
2. Click **"ADD USERS"**
3. Enter your Gmail address(es)
4. Click **"SAVE AND CONTINUE"**

Step 4: Create OAuth 2.0 Credentials

1. Go to **"APIs & Services"** → **"Credentials"**
2. Click **"CREATE CREDENTIALS"** → **"OAuth client ID"**

Client Configuration

Field	Value
Application type	Web application
Name	Your app name

Authorized Redirect URIs

For **Postman testing**:

text

```
https://oauth.pstmn.io/v1/callback
```

For **Production**:

text

```
https://yourdomain.com/oauth2callback
```

For **Local development**:

text

```
http://localhost:8080
http://localhost:8000[citation:1]
```

⚠ Critical: The redirect URI must match **exactly** - including `https://`, no extra slashes, no trailing spaces.

1. Click **"CREATE"**
2. **Save your Client ID and Client Secret** immediately

OAuth Scopes Reference

YouTube Data API Scopes

Scope	Description	Access Level
<code>https://www.googleapis.com/auth/youtube</code>	Manage your YouTube account	Full management
<code>https://www.googleapis.com/auth/youtube.force-ssl</code>	See, edit, and permanently delete videos, ratings, comments, captions	Full with SSL
<code>https://www.googleapis.com/auth/youtube.upload</code>	Manage your YouTube videos	Upload only
<code>https://www.googleapis.com/auth/youtube.readonly</code>	View your YouTube account	Read-only
<code>https://www.googleapis.com/auth/youtubepartner</code>	View/manage assets and associated content	Partner
<code>https://www.googleapis.com/auth/youtube.channel-memberships.creator</code>	See active channel members	Membership read

YouTube Analytics Scopes

Scope	Description
https://www.googleapis.com/auth/yt-analytics.readonly	View YouTube Analytics reports
https://www.googleapis.com/auth/yt-analytics-monetary.readonly	View monetary and non-monetary reports

Best Practice: Request only the scopes your app actually needs. Requesting excessive scopes reduces the likelihood of user consent.

Postman Configuration

Step 1: Open OAuth 2.0 Configuration

1. Open Postman → Create **New Request**
2. Go to **Authorization** tab
3. **Auth Type:** Select **OAuth 2.0**

Step 2: Configure New Token

Field	Value
Token Name	YouTube Token (or any name)
Grant Type	Authorization Code
Callback URL	https://oauth.pstmn.io/v1/callback
Auth URL	https://accounts.google.com/o/oauth2/auth
Access Token URL	https://oauth2.googleapis.com/token
Client ID	Your Google Client ID
Client Secret	Your Google Client Secret
Scope	https://www.googleapis.com/auth/youtube.force-ssl
Client Authentication	Send as Basic Auth header

Step 3: Get Access Token

1. Click "**Get New Access Token**"
2. Google login popup appears
3. Select your Google account

4. Review and **"Allow"** permissions
5. Click **"Use Token"**

Step 4: Verify Token Works

Test Request:

http

```
GET https://www.googleapis.com/youtube/v3/channels?part=snippet&mine=true
Authorization: Bearer {your_access_token}
```

Expected Response:

json

```
{
  "kind": "youtube#channelListResponse",
  "items": [
    {
      "id": "UCXXXXXXX",
      "snippet": {
        "title": "Your Channel Name",
        "description": "..."
      }
    }
  ]
}
```

API Operations Guide

List Your Channels

http

```
GET https://www.googleapis.com/youtube/v3/channels?part=snippet,statistics&mine=true
```

Parameters:

Parameter	Description
<code>part</code>	Required - comma-separated properties (snippet, statistics, contentDetails)
<code>mine</code>	Set to <code>true</code> for authenticated user's channels

List All Playlists

http

```
GET https://www.googleapis.com/youtube/v3/playlists?part=snippet&mine=true
```

Response includes pagination:

json

```
{
  "items": [...],
  "nextPageToken": "CAUQAA",
  "pageInfo": {
    "totalResults": 16,
    "resultsPerPage": 5
  }
}
```

To get next page, add `&pageToken={nextPageToken}`.

List Videos in a Playlist

http

```
GET https://www.googleapis.com/youtube/v3/playlistItems?part=snippet&playlistId={PLAYLIST_ID}
```

Insert Video into Playlist

http

```
POST https://www.googleapis.com/youtube/v3/playlistItems?part=snippet
Authorization: Bearer {access_token}
Content-Type: application/json
```

Body:

json

```
{
  "snippet": {
    "playlistId": "PLAYLIST_ID",
    "resourceId": {
      "kind": "youtube#video",
      "videoId": "VIDEO_ID"
    }
  }
}
```

Delete Video from Playlist

http

```
DELETE https://www.googleapis.com/youtube/v3/playlistItems?id={PLAYLIST_ITEM_ID}
```

Search Videos

http

```
GET https://www.googleapis.com/youtube/v3/search?part=snippet&q={SEARCH_TERM}&maxResults=10
```

Note: Search requires an API key or OAuth token. Without authentication, only public results return.

Upload a Video (2-Step Process)

Step 1: Get Upload URL (Resumable)

http

```
POST https://www.googleapis.com/upload/youtube/v3/videos?part=snippet,status
Authorization: Bearer {access_token}
Content-Type: application/json; charset=UTF-8
X-Upload-Content-Length: {VIDEO_SIZE}
X-Upload-Content-Type: {VIDEO_MIME_TYPE}
```

Body:

json

```
{
  "snippet": {
    "title": "Video Title",
    "description": "Video description",
    "tags": ["tag1", "tag2"],
    "categoryId": "22"
  },
  "status": {
    "privacyStatus": "public"
  }
}
```

Response includes `Location` header with upload URL.

Step 2: Upload Video File

http

```
PUT {upload_url_from_step_1}
Content-Type: {VIDEO_MIME_TYPE}
Content-Length: {VIDEO_SIZE}

[VIDEO_BINARY_DATA]
```